

更聪明的算法：三条方向的攻坚结果

贴边定向源的严格解法、检测次数的下界、顺路清除的否定、
以及”完成度 vs 时间”的真实前沿
——四池 120 例联合验证，含全部负结果

2026 高教社杯全国大学生数学建模竞赛 B 题

2026 年 9 月 12 日

摘要

本轮针对三条指定方向做了实质性算法改造，并在四个互不重叠的独立算例池（种子 62000+、51000+、12000+、30000+，各 30 例，共 120 例约 1560 个干扰源）上联合验证。选型一律以最差池为准。

（一）贴边定向源——解决到几何极限。重新刻画了问题：机器人在边界点听到源后，沿射线走不改变方位（拿不到第二条独立方位），垂直走可能越界（落到背光侧）。正确做法是对横向偏移量本身做二分——因为受光半平面必然包含听到点，偏移趋于零时必然仍可听，所以这套二分收敛性有保证。配合末段栅格穷举清除（间距 15 m， $15/\sqrt{2} = 10.6 \text{ m} < 20 \text{ m}$ 清除半径，因此只要 σ 覆盖真值就必然命中），把四个池的漏源从 3 个降到 1 个，而且平均时间还略降（885.3 s $\rightarrow$ 880.7 s）。

仅剩的 1 例经几何核算属于问题本身的极限而非算法缺陷：该源位于 (1767, -219)、辐射方向与朝外径向的夹角余弦为 0.996（几乎完全朝外），其域内受光区只有 0.011 km^2 ，占全域的 0.11%，是一条 27 m 宽的月牙。要保证覆盖它需要在整条 11.3 km 的边界上每 25 m 布一个站位（约 450 站），不可行。

（二）减少每站检测频道数——已做到接近下界。实测每站检测次数随频道逐个解算由 20 降到 9~16。新增的”站位附近已全部认证则不再测量”规则在逻辑上严格成立（认证扫描是精确的），但实测几乎不触发——因为认证本身已接近信息下界：每个候选点只需 3~4 条环绕读数，而格点扫描正好提供这么多。继续压缩只能靠削弱认证本身，那是用完成率换时间。

（三）顺路清除——实测为负优化。关闭时完成率 1.0000、540.3 s；开启（半径 520 m）后降到 0.9946、573.3 s。原因是清除任务自身的归航与末段图案带来的额外移动超过了它省下的那趟绕行，并且打断了扫描的发现进度。

（四）混合策略的真正答案。论文配置的 24 个站位里有 17 个是认证搜索补出来的，而认证搜索把站位放到”证书最弱”处——那正是定向源可能藏身之处。换句话说认证搜索兼任了定向源发现器，这是它真正的价值。把认证搜索去掉换成”更密的纯格点扫描”，时间确实降到 533.8 s，但最差完成率从 0.9978 掉到 0.9833（120 例中 14 例失败）。加贴边环（6~16 站）只能把最差补到 0.9905，代价是时间回到 583 s。

最终结论（诚实版）：在完成度完整的前提下，本轮把”漏源”从 3 个压到 1 个、时间基本持平；而”完成度完整”与”时间大幅压缩”没有同时达成——这是一个真实的前沿：约 880 s 对应 0.9978，约 534 s 对应 0.9833。本报告给出完整前沿与每一项尝试的判定，供权衡。

目录

1 验证协议：四池联合，以最差池为准

2	方向一：贴边定向源的严格解法	3
2.1	问题的完整刻画	3
2.2	正确构造：对横向偏移量二分	3
2.3	第二件工具：末段栅格穷举清除	4
2.4	效果（四池 120 例 A/B）	4
2.5	仅剩的 1 例：几何极限，不是算法缺陷	4
3	方向二：减少每站检测频道数	5
3.1	实测：检测次数已随频道解算自然下降	5
3.2	新规则” 站位附近已全部认证则不再测量”	5
4	方向三：顺路清除（与巡回规划的混合）	5
4.1	动机	5
4.2	实现与实测：负优化	6
5	混合策略的真正答案：认证搜索兼任定向源发现器	6
5.1	发现	6
5.2	贴边环：方向正确但不足以补齐	6
6	最终前沿：完成度与时间的真实交换	7
7	完整尝试清单（本轮）	7
8	复现方式	8

1 验证协议：四池联合，以最差池为准

本轮的所有结论都在四个互不重叠的池上取平均，并以最差池作为选型依据：

池	种子	规模
A	62000–62029	30 例
B	51000–51029	30 例
C	12000–12019	30 例
D	30000–30029	30 例

为什么必须这样。本项目曾三次被单一池误导：

1. 第一轮”下降 44%”的假提升（算例池被复用）；
2. 第二轮”50% 相关性最差”的假异常（误差分布标准差不同，对照不公平）；
3. 第三轮本轮开始时： $s = 900$ 在调参池（30000+）上是 $1.0000 / 540.3$ s，在池 A（62000+）上却只有 0.9928；而 $s = 1000$ 在池 A 上是 $1.0000 / 539.3$ s，在调参池上只有 0.9831。两个方向都被误导过。

因此本轮不再使用”调参池”，所有选型直接看四池汇总。

2 方向一：贴边定向源的严格解法

2.1 问题的完整刻画

前三轮已经把现象描清楚了，这里给出严格的因果链：

1. 一个贴在边界、朝外辐射的定向源，其域内受光区是一条薄月牙；
2. 机器人只能在月牙里的某点 p 听到它一次；
3. 只有一条方位 $\Rightarrow$ 定位区域无界（实测 `bounded=False`） $\Rightarrow$ 不生成清除任务；
4. 要拿到第二条方位就必须换观测点，而：
 - 沿射线走：方位不变（源就在这条射线上），拿不到独立的第二条方位；
 - 垂直走：可能越出受光半平面，返回 `no_signal`。

前两轮的成功尝试正好落在这两个陷阱里：沿射线 550 m 单跳越过了源落到背光侧；沿射线 200 m 爬行虽然拿到了第二条方位，但两条方位只差 1.3° ，交会区域仍然无界（ $\sigma = 670$ m）。

2.2 正确构造：对横向偏移量二分

关键几何事实：

偏移二分必然收敛

机器人在 p 听到源，说明 p 落在源的受光半平面 H 内。取 p 处垂直于视线的方向，偏移 δ 后得到 p' 。当 $\delta \rightarrow 0$ 时 $p' \rightarrow p \in H$ ，因此存在 $\delta_0 > 0$ ，使得所有 $\delta < \delta_0$ 的偏移点仍在 H 内、仍能听到源。于是按 $\delta = 400, 250, 150, 90, 55, 33, 20$ m 依次尝试（两侧都试），必在有限步内成功——这不是启发式，而是由半平面的开性保证的收敛构造。

代码实现为 `relocate` 任务（`_relocate_point / _relocate_advance`）。若全部偏移档位都失败，则沿射线前爬 `relocate_crawl_step = 200` m 再重新从大偏移试起——此时横向偏移的绝对尺度自动变小，等价于把二分继续下去。

实现中的一个真实 bug（值得记录）

初版把状态机的推进写在 `_relocate_point()` 里，而这个函数是在构建任务列表时被调用的；`_build_tasks()` 又在每次重规划（即每个动作之后）被调用。结果是：状态机在没有任何候选点被实际访问的情况下就跑完了，6 个优质候选点（离源 40~90 m 且受光）一个都没去。现象是“机制明明生成了正确点位，机器人却始终停在原地”。修法：查询与推进分离——`_relocate_point()` 只读取状态，状态在任务执行完之后由 `_relocate_advance()` 推进。

2.3 第二件工具：末段栅格穷举清除

诊断（池 C / seed 12010 / ch12）：源在 $r = 1799$ 、定向，机器人已把它定位到 $\sigma = 42$ m，但 14 次清除全部失败并用尽了硬上限，源被漏掉。原因是末段图案取半径 $0.55\sigma \approx 23$ m、两圈 23 m 与 46 m，图案点之间的空隙可能整片落在 20 m 清除半径之外。

有保证的兜底

当估计已足够好（ σ 不超过定位门槛）而常规末段图案连续失败后，在 σ 圆盘上做间距 $g = 15$ m 的方形栅格穷举清除。栅格中圆盘内任一点到最近栅格点的距离不超过 $g/\sqrt{2} = 10.6$ m < 20 m，因此只要真值落在 σ 圆盘内就必然命中（这由问题一的定位区域理论保证）。代价最坏约 $25 \times 3 = 75$ s，只在最后手段触发。

2.4 效果（四池 120 例 A/B）

配置	最差完成率	平均时间/s	失败算例	漏源数
本轮改动前	0.9954	885.3	3/120	3
本轮改动后	0.9978	880.7	1/120	1

漏源减少 67%，时间还略降——因为这两件工具让机器人不必再反复出车去“再试一次”，从而省下了无效往返。

2.5 仅剩的 1 例：几何极限，不是算法缺陷

```

1 source (1767, -219) r = 1780 m azimuth = 352.9 deg
2 radiation direction 348 deg ; dot(dir, outward radial) = 0.996 <-- 几乎完全朝外
3 lit area inside the arena: 0.11 % (0.011 km^2 of 10.18 km^2)
4 deepest lit point on the rim is 27 m beyond the boundary plane

```

代码 1 seed 51029 / ch2 的几何核算

也就是说：该源的可探测区域是一条 **27 m** 宽的月牙。要保证覆盖它，需要在整条 11.3 km 的边界上每 25 m 布一个站位，约 **450 个站位**；而当前每例的总时间预算是 28800 s（单源 360000 s），450 站仅移动就需 $450 \times 120 = 54000$ s。这在时间预算上不可行，因此这一例属于问题固有的测度极小的构型，不应算作算法缺陷。本文据此把可达到的完成率上界记作 $1 - 0.064\% = 0.9994$ 。

3 方向二：减少每站检测频道数

3.1 实测：检测次数已随频道解算自然下降

单例剖析（seed 30000，每站的状态分布与本次检测次数）：

站点序	未解算	已听到	已定位	已清除	本站检测次数
1（入场普查）	15	5	0	0	19
2	14	2	4	0	16
3	14	0	5	1	14
4	12	2	5	1	14
5	11	2	6	1	12
...					
中后期各站	10	0~2	0~5	3~10	10~12

检测次数由 20 降到 9~16，说明”已清除 / 已认证”的频道确实被跳过了。

3.2 新规则” 站位附近已全部认证则不再测量”

逻辑是严格的：对”从未听到”的频道 c ，在某站位 p 测一次的全部价值，在于为 p 的认证半径邻域内尚未排除的候选点提供读数；若该邻域**全部已排除**，测量不会改变任何结论。判定用认证扫描已维护的增量结构，代价 $O(\text{邻域点数}) \approx 800$ 次整数比较，相对一次 6 s 的检测可忽略。

实测：几乎不触发（405 次调用，0 次跳过）。原因很清楚——认证本身已接近信息下界：每个候选点需要 3~4 条环绕读数，而间距 900~1000 m 的格点恰好只提供这么多。**再压缩只能削弱认证本身**。

4 方向三：顺路清除（与巡回规划的混合）

4.1 动机

纯扫描配置的时间构成（16 例，6795 s/例 = 543 s/源）：

任务类	次数/例	行程/m	占总行程	占总时间
普查站位	18.6	11884	53.7%	50.5%
清除	12.7	9060	40.9%	38.0%
补测站位	6.8	2487	11.2%	13.6%
入场普查	1.0	0	0%	1.8%

清除行程已成为最大的单项（12.7 个源、平均每个 713 m）。但清除必须走到离源 20 m 以内，”必须走”的部分省不掉，能省的是从扫描路线专门拐出去再拐回来的往返。

4.2 实现与实测：负优化

配置	完成率	最差	平均时间/s	行程/m
关闭（基线）	1.0000	1.0000	540.3	22343
开启，半径 350 m	0.9972	0.9333	544.4	22741
开启，半径 520 m	0.9946	0.9333	573.3	24935
开启，半径 750 m	0.9946	0.9333	599.3	27091

完成率与时间同时变差。原因是机会式清除打断了扫描，而 `clear_channel` 自身的归航与末段图案带来的额外移动超过了它省下的那趟绕行；同时推迟了扫描的发现进度（与上一轮”分阶段调度”失败的机理同源）。

结论

”顺路清除”在本题不成立。清除不是一个可以”顺手”完成的动作：它内部含有归航、图案、复测等一串子过程，其代价远大于”走到那里”本身。这是本项目第二次在”搜索—清除耦合”上得到负结果，两次的机理是一致的。

5 混合策略的真正答案：认证搜索兼任定向源发现器

5.1 发现

论文配置的 24 个站位里，只有约 7 个来自格点，其余 17 个由认证搜索补出。把认证搜索整个去掉、改用更密的格点扫描，结果如下（四池各 30 例，按最差池判定）：

配置	站位数	最差完成率	时间/s	失败	各池完成率（A/B/C/D）
P4 论文（ $s=1100$ + 认证搜索）	24.0	0.9978	880.7	1/120	1.000 / 0.998 / 1.000 / 1.000
$s=1000$ 纯扫描	12.8	0.9833	533.8	14/120	1.000 / 0.983 / 0.990 / 0.986
$s=900$ 纯扫描	18.3	0.9751	554.5	17/120	0.993 / 0.982 / 0.975 / 1.000
$s=950$ 纯扫描	12.9	0.9803	515.1	21/120	0.987 / 0.980 / 0.986 / 0.986
$s=1000$ + 认证搜索（预算 6）	13.0	0.9833	539.4	14/120	—

关键观察：认证搜索的补站在 $s = 1100$ 时有 17 个，在 $s = 1000$ 时几乎为 0。为什么？因为间距 $\leq$ 认证半径 958 m 时，候选点所在格点三角形的三个顶点都在认证半径内，格点自身就能完成认证；而 $s = 1100$ 时不行，于是补站很多。

那为什么 $s = 1100+17$ 补站反而完成率最高？因为那 17 个补站是被”证书最弱”这一信息驱动放置的——而”证书最弱”的位置，恰恰就是定向源最可能藏身的位置。认证搜索兼任了定向源发现器。

5.2 贴边环：方向正确但不足以补齐

贴边朝外的定向源，其受光月牙必与半径 1750 m 的圆环相交，所以在环上均匀补几个站位是”对症”的。实测：

配置	平均站位数	最差完成率	平均时间/s
s=1000，无贴边环	12.8	0.9833	533.8
s=1000，贴边环 6	18.7	0.9879	558.3
s=1000，贴边环 9	21.3	0.9849	561.9
s=1000，贴边环 12	24.2	0.9869	582.4
s=1000，贴边环 16	28.0	0.9905	583.2
P4 + 贴边环 6	—	0.9978	906.9

贴边环把最差完成率从 0.9833 抬到 0.9905，但补不回 P4 的 **0.9978**，而且站位数涨到 28——说明均匀补环不如”信息驱动的定向补站”。

6 最终前沿：完成度与时间的真实交换

配置	最差完成率	平均时间/s	失败算例
P4 + relocate + exhaustive（推荐）	0.9978	880.7	1/120
P4 + 贴边环 6	0.9978	906.9	1/120
s=1000 + 贴边环 16	0.9905	583.2	12/120
s=1000 纯扫描	0.9833	533.8	14/120
s=950 纯扫描	0.9803	515.1	21/120

诚实结论

”完成度完整”与”时间大幅压缩”没有同时达成。本轮在完成度上取得了实质进步（漏源 3 → 1，且时间持平），但把时间从 880 s 压到 530~580 s 的代价是**最差完成率下降 0.7~1.5 个百分点**，且 120 例中会出现 12~14 例未能清完。

没有找到既能保持 0.9978 以上、又能把时间压到 600 s 以内的配置。从数据看，这不是调参不足，而是结构性的：P4 的 17 个认证补站既是”认证”也是”定向源发现”，去掉它们就同时失去了发现能力。若要在二者间取得更优的折中，唯一未探索的方向是把”发现”与”认证”明确分成两个目标函数来联合布站（当前是单一目标：单位代价新认证的候选点数）。

7 完整尝试清单（本轮）

序号	尝试	结果	判定	依据
R1	relocate: 横向偏移二分	单条方位的频道拿到第二条方位	有效	4 例已知失败全修复
R2	relocate 状态机修正	候选点从”生成即丢”变为”逐点访问”	关键修复	单例追踪
R3	relocate 偏移由小到大	局部行程减半但总行程反增、完成率降	负优化	24 例对照
R4	relocate 预算（档位数/爬行次数）	完全无差别	无效	24 例对照
R5	exhaustive: 末段栅格穷举清除	定位到 42 m 却清不掉的源被清掉	有效	单例 + 四池 A/B
R6	硬上限 14 → 24	配合 R5 使用	有效	同上
R7	skip_certified: 附近已认证则不测	逻辑严格，但几乎不触发	无效（近下界）	405 次调用 0 次跳过

序号	尝试	结果	判定	依据
R8	顺路清除 (半径 350/520/750 m)	完成率与时间同时变差	负优化	24 例对照
R9	纯扫描替代认证搜索 (s=900/950/1000)	时间 -38%，最差完成率 -1.5 pp	权衡，不采用	四池 120 例
R10	贴边环 6/9/12/16	最差 0.9833 → 0.9905，但补不回 0.9978	部分有效	四池 120 例
R11	认证预算 (0/2/4/6/12/40)	几乎无差别	无效	多轮对照
R12	清除偏置 × 定位门槛 (12 组)	cb500/l5400 最优	参数确认	24 例 × 12 组

8 复现方式

```
1 cd code
2 python multi_pool.py 30      # 四池 120 例：候选配置总览（本报告主表）
3 python ab_relocate.py 30     # 本轮两个新机制的 A/B（漏源 3 -> 1）
4 python rim_ring_scan.py 30   # 贴边环规模扫描
5 python relocate_budget.py 24 # relocate 预算
6 python measure_profile.py 30000 # 每站检测次数剖析
7 python find_missed.py 40     # 列出所有漏源及其几何特征
```

配置：论文默认使用 make_data.P4（现已包含本轮的两个新机制）；P4_PERFECT 与 P4_FAST 是”省时”预设，其完成率代价见第 6 节，按需选用。